



시험에 나오는것만 공부한다!

시나공시리즈

기출문제

2024년 3회 정보처리기사 실기



정보처리기사 실기 시험은 한국산업인력공단에서 문제를 공개하지 않아 문제 복원에 많은 어려움이 있습니다. 다음에 제시된 문제는 시험을 치른 학생들의 기억을 토대로 복원한 것이므로, 일부 내용이나 문제별 배점이 실제 시험과 다를 수 있음을 알립니다.

저작권 안내

이 자료는 시나공 카페 회원을 대상으로 하는 자료로서 개인적인 용도로만 사용할 수 있습니다. 허락 없이 복제하거나 다른 매체에 옮겨 실을 수 없으며, 상업적 용도로 사용할 수 없습니다.

*** 수험자 유의사항 ***

1. 시험 문제지를 받는 즉시 응시하고자 하는 종목의 문제지가 맞는지를 확인하여야 합니다.
2. 시험 문제지 총면수·문제번호 순서·인쇄상태 등을 확인하고, 수험번호 및 성명을 답안지에 기재하여야 합니다.
3. 문제 및 답안(지), 채점기준은 일절 공개하지 않으며 자신이 작성한 답안, 문제 내용 등을 수험표 등에 이기(옮겨 적는 행위) 등은 관련 법 등에 의거 불이익 조치 될 수 있으니 유의하시기 바랍니다.
4. 수험자 인적사항 및 답안작성(계산식 포함)은 흑색 기구만 사용하되, 동일한 한 가지 색의 필기구만 사용하여야 하며 흑색을 제외한 유색 필기구 또는 연필류를 사용하거나 2가지 이상의 색을 혼합 사용하였을 경우 그 문항은 0점 처리됩니다.
5. 답란(답안 기재란)에는 문제와 관련 없는 불필요한 낙서나 특이한 기록사항 등을 기재하여서는 안되며 부정의 목적으로 특이한 표식을 하였다고 판단될 경우에는 모든 문항이 0점 처리됩니다.
6. 답안을 정정할 때에는 반드시 정정부분을 두 줄(=)로 그어 표시하여야 하며, 두 줄로 굿지 않은 답안은 정정하지 않은 것으로 간주합니다.
7. 답안의 한글 또는 영문의 오타자는 오답으로 처리됩니다. 단, 답안에서 영문의 대·소문자 구분, 띄어쓰기는 여부에 관계 없이 채점합니다.
8. 계산 또는 디버깅 등 계산 연습이 필요한 경우는 <문 제> 아래의 연습란을 사용하시기 바라며, 연습란은 채점대상이 아닙니다.
9. 문제에서 요구한 가지 수(항수) 이상을 답란에 표기한 경우에는 답안기재 순으로 요구한 가지 수(항수)만 채점하고 한 항에 여러 가지를 기재하더라도 한 가지로 보며 그 중 정답과 오답이 함께 기재란에 있을 경우 오답으로 처리됩니다.
10. 한 문제에서 소문제로 파생되는 문제나, 가지수를 요구하는 문제는 대부분의 경우 부분채점을 적용합니다. 그러나 소문제로 파생되는 문제 내에서의 부분 배점은 적용하지 않습니다.
11. 답안은 문제의 마지막에 있는 답란에 작성하여야 합니다.
12. 부정 또는 불공정한 방법(시험문제 내용과 관련된 메모지사용 등)으로 시험을 치른 자는 부정행위자로 처리되어 당해 시험을 중지 또는 무효로 하고, 2년간 국가기술자격검정의 응시자격이 정지됩니다.
13. 시험위원이 시험 중 신분확인을 위하여 신분증과 수험표를 요구할 경우 반드시 제시하여야 합니다.
14. 시험 중에는 통신기기 및 전자기기(휴대용 전화기 등)를 지참하거나 사용할 수 없습니다.
15. 국가기술자격 시험문제는 일부 또는 전부가 저작권법상 보호되는 저작물이고, 저작권자는 한국산업인력공단입니다. 문제의 일부 또는 전부를 무단 복제, 배포, 출판, 전자출판 하는 등 저작권을 침해하는 일체의 행위를 금합니다.

※ 수험자 유의사항 미준수로 인한 채점상의 불이익은 수험자 본인에게 전적으로 책임이 있음

문제 1 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
public class Main{
    static String[] x = new String[3];
    static void func(String[] x, int y) {
        for(int i = 1; i < y; i++) {
            if(x[i-1].equals(x[i])) {
                System.out.print("O");
            }
            else {
                System.out.print("N");
            }
        }
        for (String z : x) {
            System.out.print(z);
        }
    }

    public static void main(String[] args) {
        x[0] = "A";
        x[1] = "A";
        x[2] = new String("A");
        func(x, 3);
    }
}
```

답 :

문제 2 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
def func(lst):
    for i in range(len(lst) // 2):
        lst[i], lst[-i-1] = lst[-i-1], lst[i]
lst = [1,2,3,4,5,6]
func(lst)
print(sum(lst[:2]) - sum(lst[1:2]))
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하기 바랍니다.

문제 3 다음 <employee>와 <project> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오. (5점)



<employee>

no	first_name	last_name	project_id
1	John	Doe	10
2	Jim	Carry	20
3	Rachel	Redmond	10

<project>

project_id	name
10	Alpha
20	Beta
10	Gamma

<SQL문>

```
SELECT count(*)
FROM employee AS e JOIN project AS p ON e.project_id = p.project_id
WHERE p.name IN (
    SELECT name FROM project p WHERE p.project_id IN (
        SELECT project_id FROM employee GROUP BY project_id HAVING count(*) < 2
    )
);
```

답 :

문제 4 3개의 페이지를 수용할 수 있는 주기억장치가 있으며, 초기에는 모두 비어 있다고 가정한다. 다음의 순서로 페이지 참조가 발생할 때, LRU 페이지 교체 알고리즘을 사용할 경우 몇 번의 페이지 부재(Page Fault)가 발생하는지 쓰시오. (5점)



페이지 참조 순서 : 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

답 :

※ 다음 여백은 연습란으로 사용하기 바랍니다.

연 습 란

문제 5 다음 설명에 해당하는 알맞은 용어를 쓰시오. (5점)



- IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법이다.
- 공격자는 송신 주소를 공격 대상지의 IP 주소로 위장하고 해당 네트워크 라우터의 브로드캐스트 주소를 수신지로 하여 패킷을 전송하면, 라우터의 브로드캐스트 주소로 수신된 패킷은 해당 네트워크 내의 모든 컴퓨터로 전송된다.
- 해당 네트워크 내의 모든 컴퓨터는 수신된 패킷에 대한 응답 메시지를 송신 주소인 공격 대상지로 집중적으로 전송하게 되는데, 이로 인해 공격 대상지는 네트워크 과부하로 인해 정상적인 서비스를 수행할 수 없게 된다.

답 :

문제 6 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
#include <stdio.h>
int func( ) {
    static int x = 0;
    x += 2;
    return x;
}

int main( ) {
    int x = 0;
    int sum= 0;
    for(int i = 0; i < 4; i++) {
        x++;
        sum += func( );
    }
    printf("%d", sum);
    return 0;
}
```

답 :

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

연 습 란

문제 7 다음 설명에 해당하는 알맞은 용어를 영문 약어 3글자로 쓰시오. (5점)



- 인터넷 등 통신 사업자의 공중 네트워크와 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선을 사용하는 것처럼 해주는 보안 솔루션이다.
- 암호화된 규격을 통해 인터넷망을 전용선의 사실망을 구축한 것처럼 이용하므로 비용 부담을 줄일 뿐만 아니라 원격지의 지사, 영업소, 이동 근무자가 지역적인 제한 없이 업무를 수행할 수 있다.
- 종류에는 IPSec, SSL 등이 있다.

답 :

문제 8 소프트웨어 공학의 디자인 패턴(Design Pattern)에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 답을 쓰시오. (5점)



- () 패턴은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴이다.
- () 패턴은 하나의 객체로 수행할 수 없는 작업을 여러 객체로 분배하면서 결합도를 최소화 할 수 있도록 도와준다.
- () 패턴에는 책임 연쇄(Chain of Responsibility), 반복자(Iterator), 옵서버(Observer) 패턴 등이 있다.

답 :

문제 9 다음 화이트박스 테스트의 검증 기준에 대한 각 번호(①~③)에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. (5점)



- ① 소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- ② 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- ③ 소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.

<보기>

㉠ 선택 ㉡ 경로 ㉢ 조건 ㉣ 분기 ㉤ 함수 ㉥ 문장 ㉦ 루프

답

- ①
- ②
- ③

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 10 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
#include <stdio.h>
struct Node {
    int value;
    struct Node* next;
};

void func(struct Node* node){
    while(node != NULL && node -> next != NULL) {
        int t = node -> value;
        node -> value = node -> next -> value;
        node -> next -> value = t;
        node = node -> next -> next;
    }
}

int main( ) {
    struct Node n1 = {1, NULL};
    struct Node n2 = {2, NULL};
    struct Node n3 = {3, NULL};
    n1.next = &n3;
    n3.next = &n2;
    func(&n1);
    struct Node* current = &n1;
    while(current != NULL){
        printf("%d", current -> value);
        current = current -> next;
    }
    return 0;
}
```

답 :

연습란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 11 다음에 제시된 URL 구조에서 <보기>의 순서대로 URL에 해당하는 번호(1~5)를 쓰시오. (5점)



<보기>

- ① query : 서버에 전달할 추가 데이터
- ② path : 서버 내의 특정 자원을 가리키는 경로
- ③ scheme : 리소스에 접근하는 방법이나 프로토콜
- ④ authority : 사용자 정보, 호스트명, 포트 번호
- ⑤ fragment : 특정 문서 내의 위치

foo://example.com:8042/over/there?name=ferret#nose

1

2

3

4

5

답

- ①
- ②
- ③
- ④
- ⑤

문제 12 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
def func(value):
    if type(value) == type(100):
        return 100
    elif type(value) == type(""):
        return len(value)
    else:
        return 20

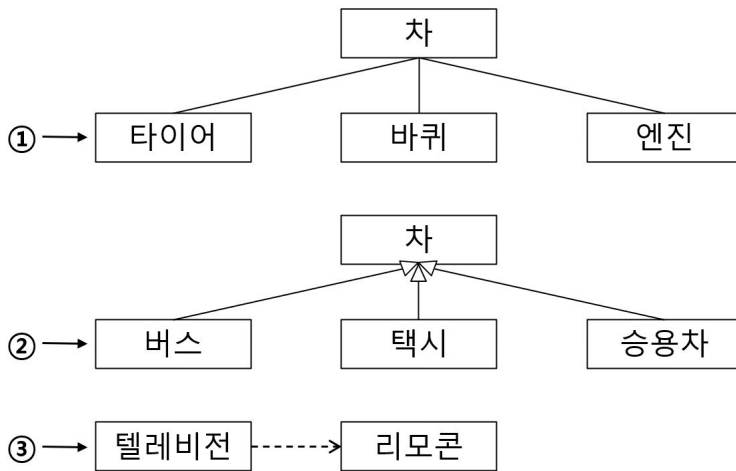
a = "100.0"
b = 100.0
c = (100, 200)
print(func(a) + func(b) + func(c))
```

답 :

※ 다음 여백은 연습란으로 사용하기 바랍니다.

연 습 란

문제 13 다음 UML 모델에서 각 번호(①~③)에 해당하는 관계를 <보기>에서 찾아 기호(㉠~㉥)로 쓰시오. (5점)



<보기>

㉠ 의존	㉡ 연관	㉢ 일반화	㉣ 집합	㉤ 포함	㉥ 실체화
------	------	-------	------	------	-------

답

- ①
- ②
- ③



연 습 란

※ 다음 여백은 연습란으로 사용하기 바랍니다.

문제 14 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
public class Main{
    public static void main(String[] args) {
        B a = new D( );
        D b = new D( );
        System.out.print(a.getX( ) + a.x + b.getX( ) + b.x);
    }
}

class B {
    int x = 3;
    int getX( ) {
        return x * 2;
    }
}

class D extends B {
    int x = 7;
    int getX( ) {
        return x * 3;
    }
}
```

답 :

시나공

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 15 키(Key)에 대한 다음 설명에서 각 번호(①~④)에 해당하는 키를 <보기>에서 찾아 기호 (㉠~㉣)로 쓰시오. (5점)



- ① 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합을 의미한다.
- ② 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용되는 속성들의 부분집합으로, 유일성과 최소성을 모두 만족시켜야 한다.
- ③ 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키를 의미한다.
- ④ 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키로, 유일성은 만족하지만, 최소성은 만족하지 못한다.

<보기>

- | | | |
|-------|-------|-------|
| ㉠ 슈퍼키 | ㉡ 외래키 | ㉢ 대체키 |
| ㉡ 후보키 | ㉣ 기본키 | ㉣ 필드키 |

답

- ①
- ②
- ③
- ④

문제 16 다음 무결성 제약 조건에 대한 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. (5점)



() 무결성은 기본 테이블의 기본키를 구성하는 어떤 속성도 Null 값이나 중복값을 가질 수 없다는 규정이다.

답 :

※ 다음 여백은 연습란으로 사용하기 바랍니다.

문제 17 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        try {
            func( );
        }
        catch (NullPointerException e) {
            sum = sum + 1;
        }
        catch (Exception e) {
            sum = sum + 10;
        }
        finally {
            sum = sum + 100;
        }
        System.out.print(sum);
    }

    static void func( ) throws Exception {
        throw new NullPointerException( );
    }
}
```

답 :

※ 다음 여백은 연습란으로 사용하기 바랍니다.

연 습 란

문제 18 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)



```
class Printer {
    void print(Integer a) {
        System.out.print("A" + a);
    }
    void print(Object a) {
        System.out.print("B" + a);
    }
    void print(Number a) {
        System.out.print("C" + a);
    }
}

public class Main {
    public static void main(String[] args) {
        new Collection<>().print( );
    }
    public static class Collection<T> {
        T value;
        public Collection(T t) {
            value = t;
        }
        public void print( ) {
            new Printer( ).print(value);
        }
    }
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 19 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (5점)



```
#include <stdio.h>
void func(int** arr, int size) {
    for(int i = 0; i < size; i++) {
        *(*arr + i) = *(*arr + i) + i % size;
    }
}

int main( ){
    int arr[] = {3, 1, 4, 1, 5};
    int* p = arr;
    int** pp = &p;
    int num = 6;
    func(pp, 5);
    num = arr[2];
    printf("%d", num);
    return 0;
}
```

답 :

문제 20 네트워크 및 인터넷과 관련된 다음 설명에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉢)로 쓰시오. (5점)



- 재난 및 군사 현장과 같이 별도의 고정된 유선망을 구축할 수 없는 장소에서 모바일 호스트(Mobile Host)만을 이용하여 구성한 네트워크이다.
- 망을 구성한 후 단기간 사용되는 경우나 유선망을 구성하기 어려운 경우에 적합하다.
- 멀티 홉 라우팅 기능을 지원한다.

<보기>

- | | | |
|------------------|------------------------|---------------------------|
| ㉠ Mesh Network | ㉡ Peer-to-Peer Network | ㉢ Virtual Private Network |
| ㉣ Ad-hoc Network | ㉤ Sensor Network | ㉥ Infrastructure Network |

답 :

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

연 습 란

기출문제 정답 및 해설

[문제 1]

OOAAA

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 소문자 **ooaaa**로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

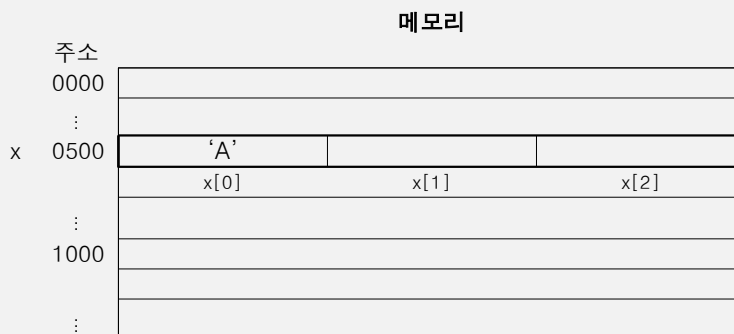
```
public class Main{
  ㉠ static String[] x = new String[3];
  ㉡ static void func(String[] x, int y) {
    ㉢ for(int i = 1; i < y; i++) {
      ㉣ if(x[i-1].equals(x[i])) {
        ㉤ System.out.print("O");
      }
      else {
        ㉥ System.out.print("N");
      }
    }
    ㉦ for (String z : x) {
      ㉧ System.out.print(z);
    }
  }

  public static void main(String[] args) {
    ㉨ x[0] = "A";
    ㉩ x[1] = "A";
    ㉪ x[2] = new String("A");
    ㉫ func(x, 3);
  }
}
```

㉠ 3개의 요소를 갖는 문자형 배열 x를 Main 클래스 어디에서든 사용할 수 있는 전역 변수로 선언한다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

㉨ x[0]에 “A”를 저장한다. 이후 그림에서 사용한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.)



㉩ x[1]에 “A”를 저장한다.

메모리

주소	0000			
	⋮			
x	0500	'A'	'A'	
		x[0]	x[1]	x[2]
	⋮			
	1000			
	⋮			

③ new 키워드로 인해 새로운 메모리 영역을 확보하여 "A"를 저장한다.

메모리

주소	0000			
	⋮			
x	0500	'A'	'A'	
		x[0]	x[1]	
	⋮			
	1000	'A'		
		x[2]		
	⋮			

④ 배열 x의 주소와 3을 인수로 하여 func() 메소드를 호출한다. ⑤번으로 이동한다.

※ 인수로 배열의 이름을 지정하면 배열의 시작 주소가 인수로 전달됩니다.

⑤ 반환값이 없는 func() 메소드의 시작점이다. ④번에서 전달한 배열의 주소와 3을 x와 y가 받는다.

⑥ 반복 변수 i가 1에서 시작하여 1씩 증가하면서, i가 y보다 작은 동안 ⑦~⑨번을 반복 수행한다.

⑦ x[i-1]과 x[i]가 같으면 ⑧번을 수행하고, 아니면 ⑨번을 수행한다.

※ A.equals(B) : A 문자열과 B 문자열을 비교하여 두 데이터가 같으면 참을, 아니면 거짓을 반환함

⑧ 영문자 O를 출력한다.

⑨ 영문자 N을 출력한다.

※ 반복문의 실행에 따른 변수들의 변화는 다음과 같다.

y	i	x[i-1]	x[i]	equals()	출력
3	1	'A'	'A'	True	○
	2	'A'	'A'	True	○○
	3				

결과 OO

⑩ x 배열의 요소 수만큼 ⑪번을 반복 수행한 후 func() 메소드를 호출했던 ④번의 다음 줄로 이동하여 프로그램을 종료한다.

· String z : 문자열 x의 각 요소가 할당될 변수

· x : 문자열 변수가 배열이며, 3개의 요소를 가지므로 각 요소를 z에 할당하면서 ⑪번을 3회 수행함

⑪ z의 값을 3번 출력한다. 즉 x 배열의 요소가 순서대로 출력된다.

결과 OOAAA

[문제 2]

3

[해설]

```

③ def func(lst):                                     // def는 Python에서 메소드를 정의하는 예약어이다. 매개 변수를 지정
                                                    한 후 메소드에 속한 실행 코드들은 콜론(:)과 여백으로 구분한다.
④     for i in range(len(lst) // 2):
⑤         lst[i], lst[-i-1] = lst[-i-1], lst[i]
① lst = [1,2,3,4,5,6]
② func(lst)
⑥ print(sum(lst[::-2]) - sum(lst[1::2]))

```

func() 메소드 정의부의 다음 줄인 4번째 줄부터 실행한다.

① 리스트 lst를 선언하면서 초기값을 지정한다. 초기값의 개수만큼 리스트의 요소가 만들어진다.

	[0]	[1]	[2]	[3]	[4]	[5]
lst	1	2	3	4	5	6
	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

② lst를 인수로 func() 메소드를 호출한다. ③번으로 이동한다.

③ func() 메소드의 시작점이다. ②번에 전달한 lst를 lst가 받는다.

④ 반복 변수 i에 0부터 lst의 길이를 2로 나눈 몫 3에서 1작은 2까지 순차적으로 저장하며 ⑤번을 반복 수행한다.

※ len() : 문자열이나 리스트의 길이를 반환함

※ // : 몫을 구하는 연산자

⑤ lst[i]에 lst[-i-1]의 값을, lst[-i-1]에 lst[i]의 값을 저장한다. 즉 lst[i]와 lst[-i-1]의 값을 서로 교환한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	lst[i]	lst[-i-1]	리스트 lst					
			[0]	[1]	[2]	[3]	[4]	[5]
			1	2	3	4	5	6
			[-6]	[-5]	[-4]	[-3]	[-2]	[-1]
0	1	6	1	2	3	4	5	6
			6					1
1	2	5	6	2	3	4	5	1
				5			2	
2	3	4	6	5	3	4	2	1
					4	3		

⑥ 'sum(lst[::-2]) - sum(lst[1::2])'의 결과인 3을 출력한다.

sum(lst[::-2]) - sum(lst[1::2])

㉠

㉡

· ㉠ : lst의 0번째 위치부터 마지막 위치까지 2씩 증가한 요소, 즉 0, 2, 4번째 요소인 6, 4, 2의 합은 12임

· ㉡ : lst의 1번째 위치부터 마지막 위치까지 2씩 증가한 요소, 즉 1, 3, 5번째 요소인 5, 3, 1의 합은 9임

※ 객체명[::증가값] : 객체의 0번째 위치에서 마지막 위치까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 가져옴

※ 객체명[초기위치:최종위치:증가값] : '초기위치'에서 '최종위치'-1까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 가져옴

결과

3

[문제 3]

1

[해설]

문제의 질의문은 하위 질의가 있는 질의문입니다. 다음과 같이 안쪽 하위 질의의 SELECT문을 먼저 검색한 다음 그 검색 결과를 본 질의의 조건에 적용하면 됩니다.

```
SELECT count(*)
FROM employee AS e JOIN project AS p ON e.project_id = p.project_id
WHERE p.name IN (
    SELECT name FROM project p WHERE p.project_id IN (
        SELECT project_id FROM employee GROUP BY project_id HAVING count(*) < 2
    )
);
```

<하위 질의 ①>

SELECT project_id	'project_id'를 조회한다.
FROM employee	<employee> 테이블을 대상으로 조회한다.
GROUP BY project_id	'project_id'를 기준으로 그룹을 지정한다.
HAVING count(*) < 2:	각 그룹의 개수가 2 미만인 그룹만을 대상으로한다.

<결과>

project_id
10
10
20

∴ 질의의 결과는 '20'이다.

<하위 질의 ②>

SELECT name	'name'을 조회한다.
FROM project p	<project> 테이블을 대상으로 조회한다. <project> 테이블의 별칭으로 <p>를 지정한다. <p>는 <project> 테이블을 의미한다.
WHERE p.project_id IN (①)	<project> 테이블의 'project_id'가 '20'인 자료만을 대상으로 한다.

<결과>

project_id	name
10	Alpha
20	Beta
10	Gamma

∴ 질의의 결과는 'Beta'이다.

<본 질의>

SELECT count(*)	조회된 레코드의 개수를 표시한다.
FROM employee AS e JOIN project AS p	• <employee> 테이블과 <project> 테이블을 대상으로 조회한다.
	• <employee>, <project> 테이블의 별칭으로 <e>, <p>를 지정한다. <e>는 <employee> 테이블을, <p>는 <project> 테이블을 의미한다.
ON e.project_id = p.project_id	<employee> 테이블의 'project_id'와 <project> 테이블의 'project_id'를 기준으로 JOIN 한다.
WHERE p.name IN (2)	<project> 테이블의 'name'이 'Beta'인 자료만을 대상으로 한다.

<결과>

no	first_name	last_name	project_id	project_id	name
1	John	Doe	10	10	Alpha
2	Jim	Carry	20	20	Beta
3	Rachel	Redmond	10	10	Gamma

∴ 질의의 결과로 표시되는 레코드의 개수는 1이다.

[문제 4]

12

[해설]

3개의 페이지를 수용할 수 있는 주기억장치이므로 아래 그림과 같이 3개의 페이지 프레임으로 표현할 수 있습니다.

참조 페이지	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
페이지 프레임	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
부재 발생	●	●	●	●		●		●	●	●	●			●		●		●		

참조 페이지가 페이지 테이블에 없으면 페이지 결함(부재)이 발생합니다. 초기에는 모든 페이지가 비어 있으므로 처음 7, 0, 1 페이지 적재 시 페이지 결함이 발생합니다. LRU 기법은 최근에 가장 오랫동안 사용되지 않은 페이지를 교체하는 기법이므로, 참조 페이지 2를 적재할 때 7을 제거한 후 2를 가져옵니다. 이러한 과정으로 모든 페이지에 대한 요구를 처리하고 나면 총 페이지 결함의 발생 횟수는 12회입니다.

[문제 5]

※ 다음 중 하나를 쓰면 됩니다.

스머프, Smurf, 스머핑, Smurfing

[문제 6]

20

[해설]

```
#include <stdio.h>
⑥ int func( ) {
⑦     static int x = 0;
⑧     x += 2;
⑨     return x;
}

int main( ) {
① int x = 0;
② int sum = 0;
③ for(int i = 0; i < 4; i++) {
④     x++;
⑤     sum += func( );
}
⑩ printf("%d", sum);
⑪ return 0;
}
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

- ① 정수형 변수 x를 선언하고, 0으로 초기화한다.
- ② 정수형 변수 sum을 선언하고, 0으로 초기화한다.
- ③ 반복 변수 i가 0부터 1씩 증가하면서 4보다 작은 동안 ④, ⑤번을 반복 수행한다.
- ④ 'x = x + 1;'과 동일하다. x의 값을 1씩 누적시킨다.
- ⑤ 'sum = sum + func();'과 동일하다. sum에 func() 함수의 결과를 누적시킨다. ⑥번으로 이동한다.
- ⑥ 정수를 반환하는 func() 함수의 시작점이다.
- ⑦ 정수형 static 변수 x를 선언하고, 0으로 초기화한다. static 변수는 처음에 한 번만 초기화되므로 값을 누적시킬 수 있다.
- ⑧ 'x = x + 2;'와 동일하다. x의 값을 2씩 누적시킨다.
- ⑨ x의 값을 함수를 호출했던 ⑤번으로 반환한다.

※ 반복문의 실행에 따른 변수들의 변화는 다음과 같다.

main() 함수			func() 함수
x	sum	i	static x
0	0	0	0
1	2	1	2
2	6	2	4
3	12	3	6
4	20	4	8

- ⑩ sum의 값 20을 출력한다.

결과 20

[문제 7]

VPN

[문제 8]

※ 다음 중 하나를 쓰면 됩니다.

행위, Behavioral

[문제 9]

① ㉠

② ㉡

③ ㉢

[문제 10]

312

[해설]

```
#include <stdio.h>
struct Node {                // Node 구조체를 정의한다.
    int value;                // 정수형 변수 value를 선언한다.
    struct Node *next;        // Node 구조체의 포인터 변수 next를 선언한다.
```

struct Node	
int value (4Byte)	struct Node *next (4Byte)
데이터를 저장할 멤버	다음 노드의 주소를 저장할 멤버

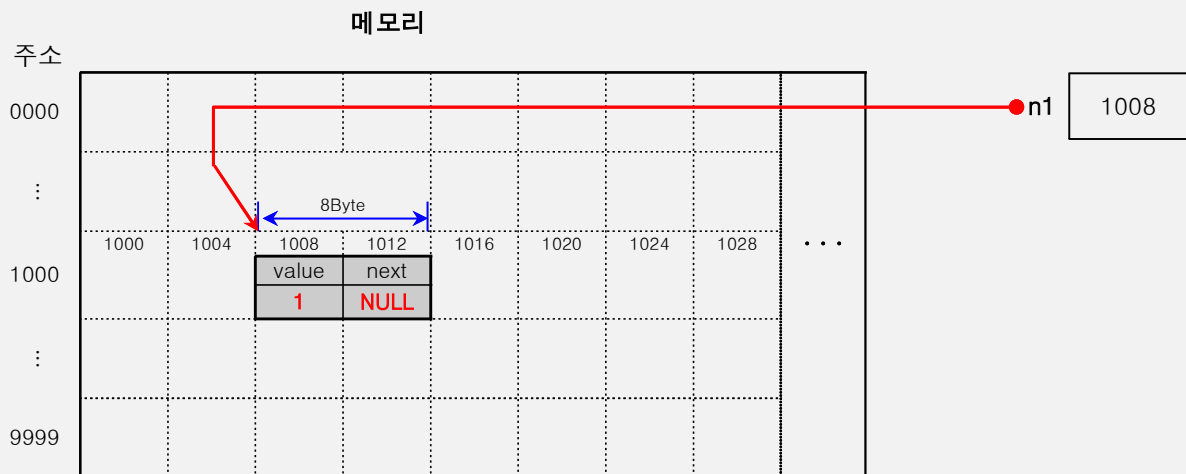
```
};
```

```
int main( ) {
    ❶ struct Node n1 = {1, NULL};
    ❷ struct Node n2 = {2, NULL};
    ❸ struct Node n3 = {3, NULL};
    ❹ n1.next = &n3;
    ❺ n3.next = &n2;
    ❻ func(&n1);
    struct Node* current = &n1;
    while(current != NULL){
        printf("%d", current -> value);
        current = current -> next;
    }
    return 0;
}
```

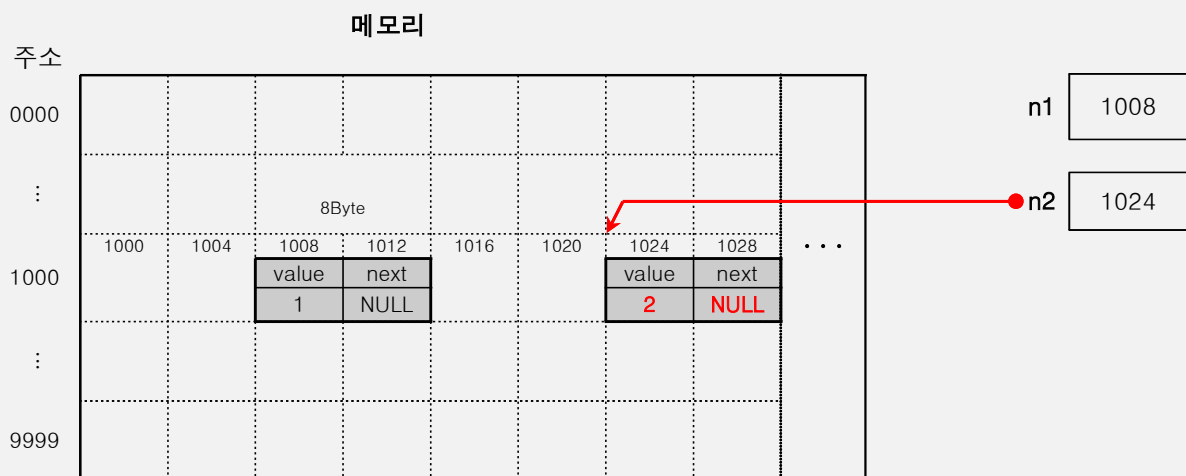
모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

❶ Node 자료형 변수 n1을 선언하고, Node의 value에 1을 Node의 next에 NULL을 저장한다. n1이 가리키고 있는 1008 번지는 Node 구조체의 크기만큼 할당된 공간이므로 n1은 1008번지 이후의 8Byte를 의미한다. (이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.)

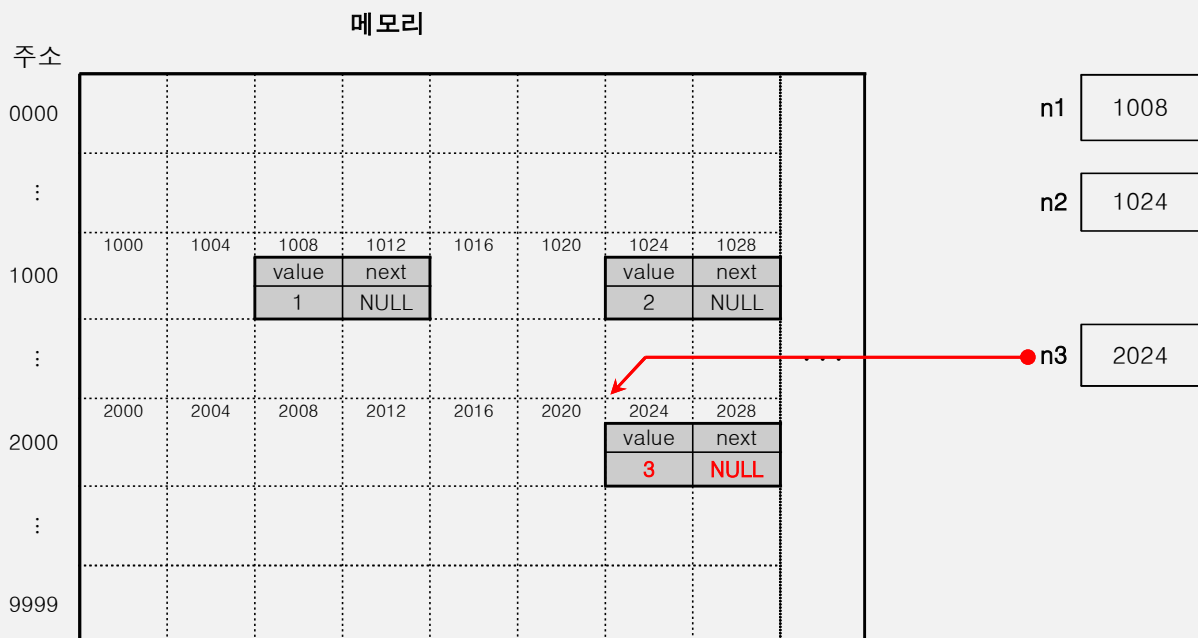
※ NULL은 값이 없음을 의미하며, next에 NULL이 저장되어 있는 것은 어떤 곳도 가리키지 않고 있음을 의미합니다.



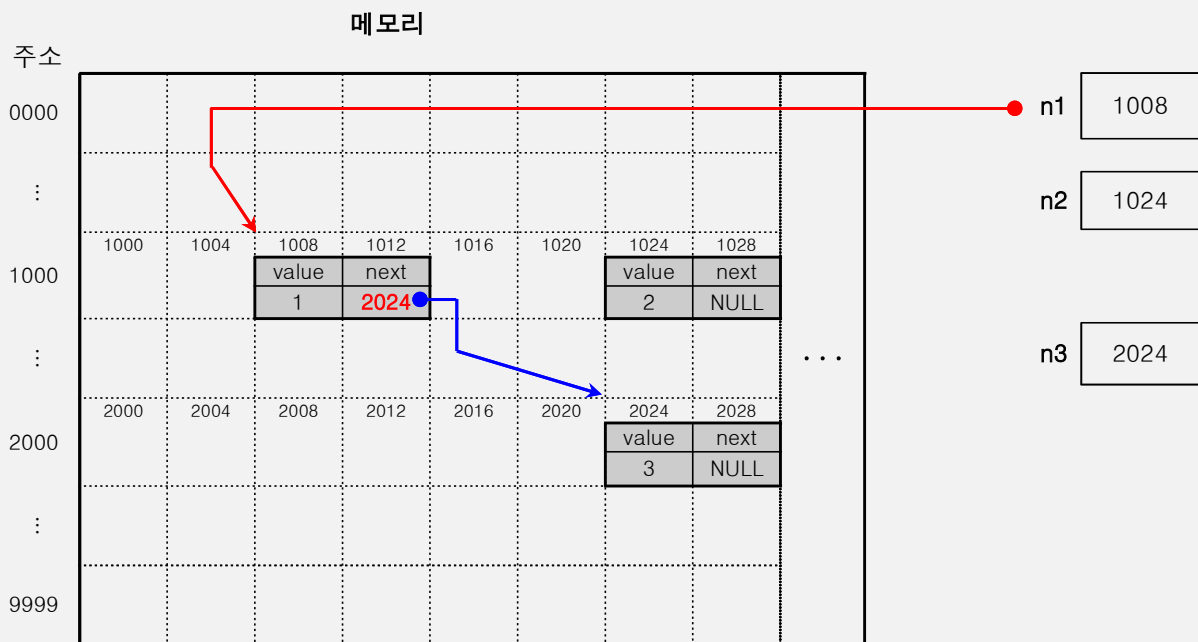
② Node 자료형 변수 n2를 선언하고, Node의 value에 2를 Node의 next에 NULL을 저장한다.



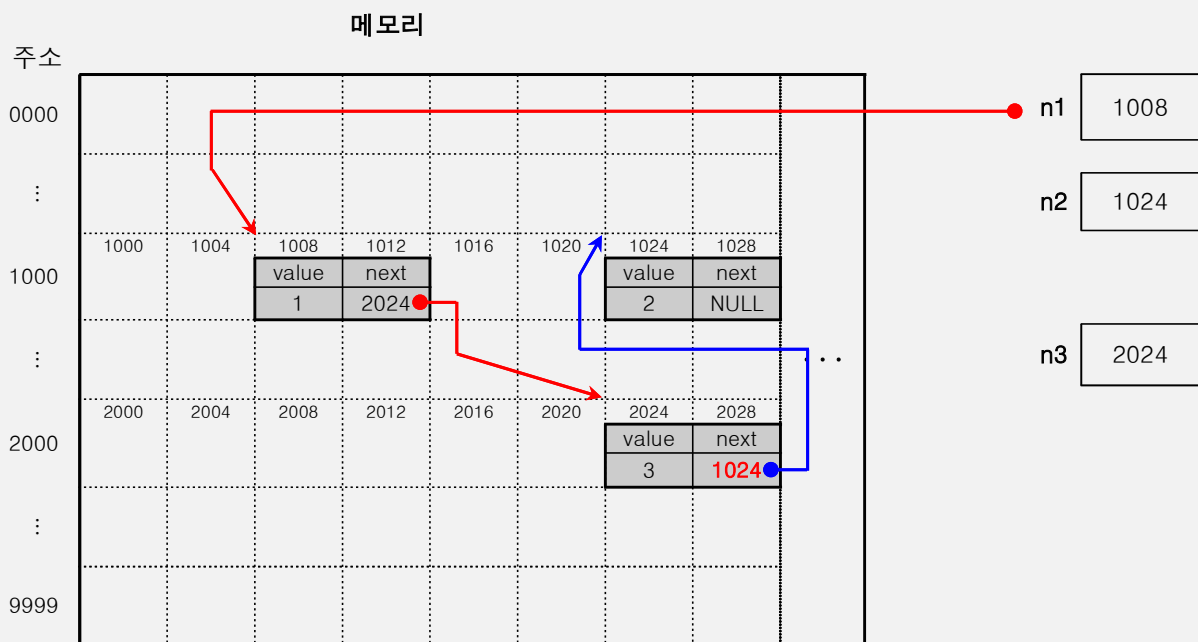
③ Node 자료형 변수 n3을 선언하고, Node의 value에 3을 Node의 next에 NULL을 저장한다.



- ④ n1의 next에 n3의 주소를 저장한다. next는 다음 노드의 주소를 가리키는 것이므로, 이제 n1의 다음 노드는 n3이다.



- ⑤ n3의 next에 n2의 주소를 저장한다. 이제 n3의 다음 노드는 n2다. 즉 이제 노드의 순서는 'n1 → n3 → n2'가 된다.



- ⑥ n1의 주소를 인수로 func() 함수를 호출한다.

```

⑦ void func(struct Node* node){
⑧ while(node != NULL && node -> next != NULL) {
⑨     int t = node -> value;
⑩     node -> value = node -> next -> value;
⑪     node -> next -> value = t;
⑫     node = node -> next -> next;
}
}

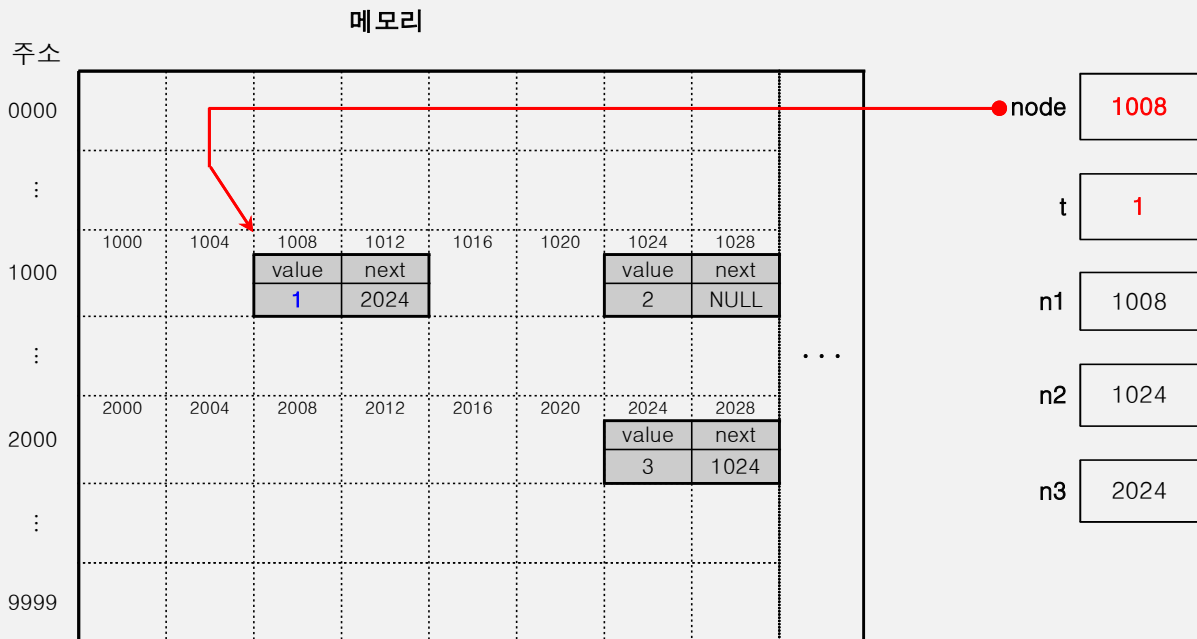
```

⑦ 반환값이 없는 func() 함수의 시작점이다. Node 자료형 포인터 변수 node가 ⑥번에서 전달한 n1의 주소를 받는다. 포인터 변수의 크기는 4Byte이므로 메모리의 어딘가에 4Byte 크기의 공간이 할당된다. 포인터 변수 node의 용도는 리스트 구조에서 첫 번째 노드의 주소를 저장하는 것이다.

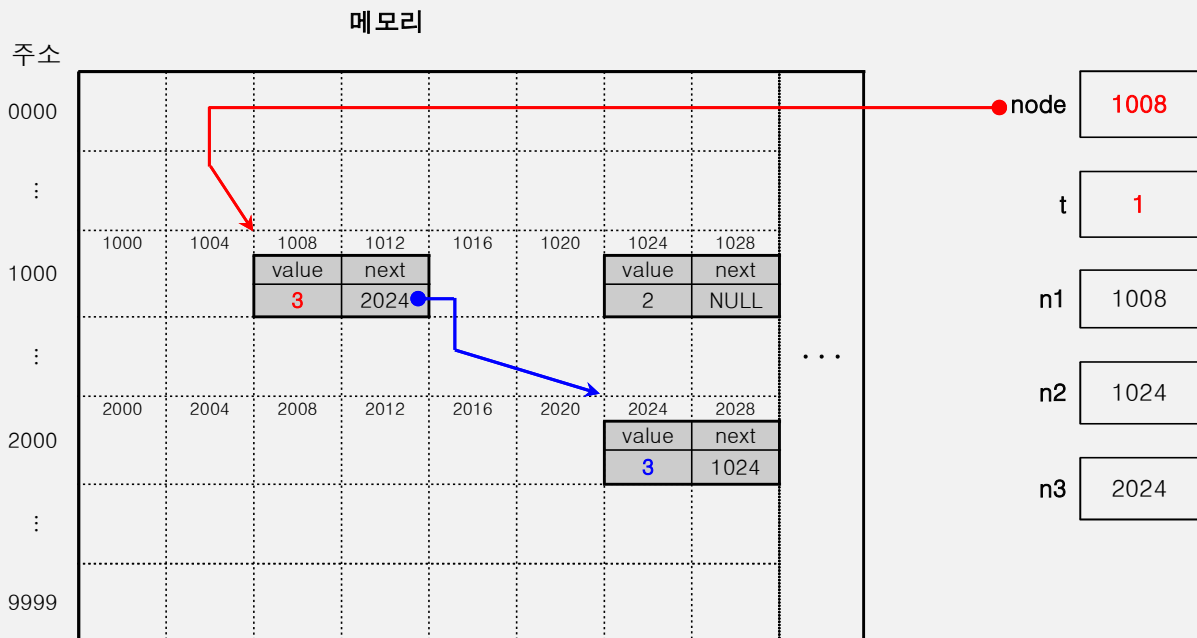
node 1008

⑧ node가 NULL이 아니고 node가 가리키는 곳의 next가 NULL이 아닌 동안 ⑨~⑫번을 반복 수행한다.

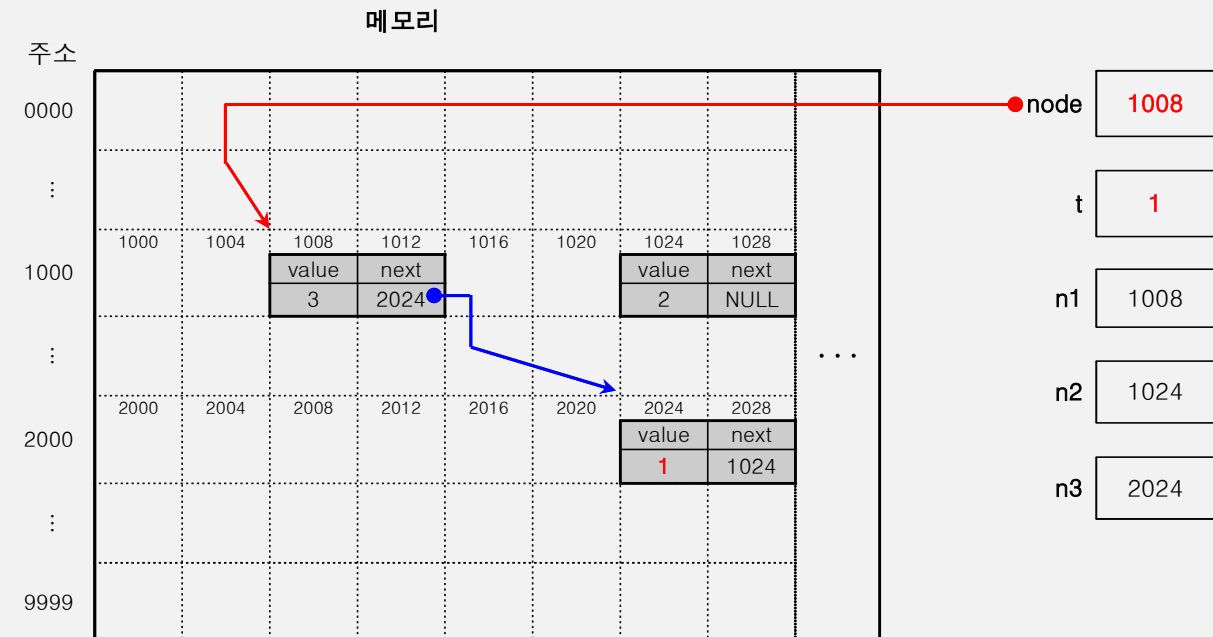
⑨ 정수형 변수 t를 선언하고, node가 가리키는 곳의 value 값인 1로 초기화한다.



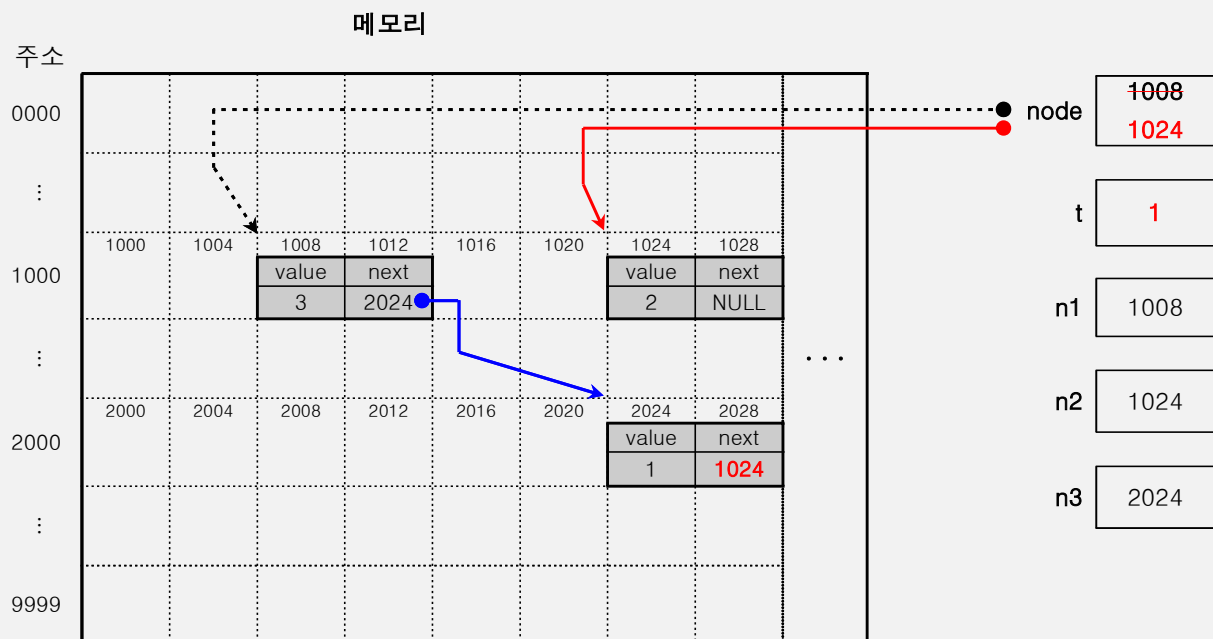
⑩ node가 가리키는 곳의 value에 node가 가리키는 곳의 next가 가리키는 곳의 value의 값인 3을 저장한다.



- ⑪ node가 가리키는 곳의 next가 가리키는 곳의 value에 t의 값 1을 저장한다.



- ⑫ node에 node가 가리키는 곳의 next가 가리키는 곳의 next의 값인 1024를 저장한다.



- ⑬ 현재 node의 값은 1024로, NULL은 아니지만 node -> next가 NULL이므로 반복문이 종료된다. 제어를 func() 함수를 호출했던 ⑭번으로 옮긴다.


```

int main( ) {
  ❶ struct Node n1 = {1, NULL};
  ❷ struct Node n2 = {2, NULL};
  ❸ struct Node n3 = {3, NULL};
  ❹ n1.next = &n3;
  ❺ n3.next = &n2;
  ❻ func(&n1);
  ❼ struct Node* current = &n1;
  ❶❷ while(current != NULL){
  ❶❸❹ printf("%d", current -> value);
  ❶❹❶ current = current -> next;
  }
  ❷❸ return 0;
}

```

❶ Node 자료형 포인터 변수 current를 선언하고, n1의 주소로 초기화한다.

주소 메모리

주소								
0000								
⋮								
1000	1000	1004	1008	1012	1016	1020	1024	1028
			value	next			value	next
			3	2024			2	NULL
⋮								
2000	2000	2004	2008	2012	2016	2020	2024	2028
							value	next
							1	1024
⋮								
9999								

current 1008

n1 1008

n2 1024

n3 2024

❶ current가 NULL이 아닌 동안 ❶❶, ❶❷번을 반복 수행한다.

❶❶ current가 가리키는 곳의 value의 값 3을 정수형으로 출력한다.

결과 3

❶❷ current에 current가 가리키는 곳의 next의 값인 2024를 저장한다.

current 2024

❶❸ current가 가리키는 곳의 value의 값 1을 정수형으로 출력한다.

결과 31

❶❹ current에 current가 가리키는 곳의 next인 1024를 저장한다.

current 1024

❶❺ current가 가리키는 곳의 value의 값 2를 정수형으로 출력한다.

결과 312

❶❻ current에 current가 가리키는 곳의 next의 값인 NULL을 저장한다.

current NULL

❶ current가 NULL이므로 반복문이 종료된다. 제어를 ❷번으로 옮긴다.

㉓ main() 함수에서의 'return 0'은 프로그램의 종료를 의미한다.

[문제 11]

- ① 4
- ② 3
- ③ 1
- ④ 2
- ⑤ 5

[문제 12]

45

[해설]

```
⑤⑪⑮ def func(value):                // def는 Python에서 메소드를 정의하는 예약어이다. 매개 변수를 지정
                                         한 후 메소드에 속한 실행 코드들은 콜론(:)과 여백으로 구분한다.
⑥⑱     if type(value) == type(100):
⑦⑲         return 100
⑧         elif type(value) == type(""):
⑨             return len(value)
⑱⑫⑯     else:
⑲⑬⑰         return 20
①         a = "100.0"
②         b = 100.0
③         c = (100, 200)
④⑩⑭⑲ print(func(a) + func(b) + func(c))
```

func() 메소드 정의부의 다음 줄인 8번째 줄부터 실행한다.

- ① 변수 a에 문자열 자료 "100.0"을 저장한다.
- ② 변수 b에 실수형 자료 100.0을 저장한다.
- ③ 변수 c에 튜플형 자료 (100, 200)을 저장한다.
- ④ a, b, c를 각각 인수로 func() 함수를 호출한 후 반환받은 값을 모두 더한 값을 출력한다. 먼저 a를 인수로 func() 함수를 호출한다. ⑤번으로 이동한다.
- ⑤ func() 메소드의 시작점이다. ④번에 전달한 a를 value가 받는다.
- ⑥ value와 100의 자료형이 같으면 ⑦번으로 이동하고, 아니면 ⑧번으로 이동한다. value의 값 "100.0"은 문자형 자료이고, 100은 정수형 자료이므로 자료형이 같지 않아 ⑧번으로 이동한다.
 - ※ type() : 인수의 자료형을 반환함
 - 정수형 : <class 'int'>를 반환함
 - 실수형 : <class 'float'>를 반환함
 - 문자형 : <class 'str'>을 반환함
- ⑧ value와 ""의 자료형이 같으면 ⑨번으로 이동하고, 아니면 else문으로 이동한다. value의 값 "100.0"과 ""은 모두 문자형으로 같으므로 ⑨번으로 이동한다.
- ⑨ 함수를 호출했던 ⑩번으로 value의 길이인 5를 반환한다.
 - ※ len() : 문자열이나 리스트의 길이를 반환함
- ⑩ b를 인수로 func() 함수를 호출한다. ⑪번으로 이동한다.
- ⑪ func() 메소드의 시작점이다. ⑩번에 전달한 b를 value가 받는다.
- ⑫ value의 값 100.0은 정수형이나 문자형 자료가 아니므로 else문이 수행되어 ⑬번으로 이동한다.
- ⑬ 함수를 호출했던 ⑭번으로 20을 반환한다.

- ⑭ c를 인수로 func() 함수를 호출한다. ⑮번으로 이동한다.
 ⑮ func() 메소드의 시작점이다. ⑭번에 전달한 c를 value가 받는다.
 ⑯ value의 값 (100, 200)은 튜플형 자료로, 실수형이나 문자형 자료가 아니므로 else문이 수행되어 ⑰번으로 이동한다.
 ⑰ 함수를 호출했던 ⑮번으로 20을 반환한다.
 ⑱ 'func(a) + func(b) + func(c)'의 결과인 45를 출력한다.

func(a) + func(b) + func(c)

㉠ ㉡ ㉢

- ㉠ : ⑨번에서 반환받은 값 5
- ㉡ : ⑮번에서 반환받은 값 20
- ㉢ : ⑰번에서 반환받은 값 20

결과 45

[문제 13]

- ① ㉠
- ② ㉢
- ③ ㉡

[해설]

- ① 차는 타이어, 바퀴, 엔진으로 구성되어 있다. 이와 같이 2개 이상의 사물이 서로 관련되어 있음을 표현하는 관계를 연관 관계라고 하며, 사물 사이를 실선으로 연결하여 표현합니다.
- ② 버스, 택시, 승용차는 차의 종류이다. 이와 같이 하나의 사물이 다른 사물에 비해 더 일반적인지를 표현하는 관계를 일반화 관계라고 하며, 구체적(하위)인 사물에서 일반적(상위)인 사물 쪽으로 속이 빈 화살표를 연결하여 표현합니다.
- ③ 텔레비전은 리모콘으로 작동한다. 이와 같이 필요에 의해 서로에게 영향을 주는 동안만 연관을 유지하는 관계를 의존 관계라고 하며, 영향을 주는 사물(이용자)이 영향을 받는 사물(제공자) 쪽으로 점선 화살표를 연결하여 표현합니다.

[문제 14]

52

[해설]

```
public class Main{
    public static void main(String[] args) {
        ❶ B a = new D( );
        ❷ D b = new D( );
        ❸❹ System.out.print(a.getX( ) + a.x + b.getX( ) + b.x);
    }
}

㉑ class B {
    int x = 3;
    int getX( ) {
        return x * 2;
    }
}

㉒ class D extends B {
    int x = 7;
    ❹❷ int getX( ) {
        ❺❸ return x * 3;
    }
}
```

㉑ B 클래스를 정의한다.

㉒ D 클래스를 정의하고 부모 클래스로 B 클래스를 지정하면서 B 클래스에 속한 변수를 상속받는다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

❶ D 클래스의 생성자를 이용하여 B 클래스의 객체 변수 a를 생성한다.

❷ D 클래스의 객체 변수 b를 생성한다.

❸ a의 getX() 메소드를 호출한 후 돌려받은 값, a의 x, b의 getX() 메소드를 호출한 후 돌려받은 값, b의 x를 모두 더한 값을 출력한다. 먼저 a의 getX() 메소드를 호출하는 ❹번으로 이동한다.

※ a.getX()는 a 객체의 자료형이 B이므로 B.getX()라고 생각할 수 있지만 ❶번에서 클래스 형 변환이 발생하였고, getX() 메소드가 자식 클래스에서 재정의되었으므로 D 클래스의 getX() 메소드가 호출됩니다. 클래스에서의 형 변환은 메소드에 대해서만 발생합니다.

❹ 정수를 반환하는 getX() 메소드의 시작점이다.

❺ x의 값이 7이므로, 21을 반환하면서 제어를 a.getX() 메소드를 호출했던 ❹번으로 옮긴다.

❻ a.x는 a 객체의 자료형이 B이고 변수는 형 변환 대상이 아니므로, B 클래스의 변수 x 값이 적용되어 3이다. 이어서 b의 getX() 메소드를 호출한다. ❷번으로 이동한다.

❼ 정수를 반환하는 getX() 메소드의 시작점이다.

❽ x의 값이 7이므로, 21을 반환하면서 제어를 b.getX() 메소드를 호출했던 ❹번으로 옮긴다.

❾ b.x는 b 객체의 자료형이 D이므로 D 클래스의 x 값인 7이다. 'a.getX() + a.x + b.getX() + b.x'의 값 52를 출력한다.

a.getX() + a.x + b.getX() + b.x

㉑ ㉒ ㉓ ㉔

· ㉑ : ❹번에서 반환받은 값 21

· ㉒ : 3

· ㉓ : ❸번에서 반환받은 값 21

· ㉔ : 7

[문제 15]

- ① ㉠
- ② ㉡
- ③ ㉢
- ④ ㉣

[문제 16]

※ 다음 중 하나를 쓰면 됩니다.

개체, Entity

[문제 17]

101

[해설]

```
public class Main {
    public static void main(String[] args) {
        ❶ int sum = 0;
        ❷ try {
            ❸ func( );
        }
        ❹ catch (NullPointerException e) {
            ❺ sum = sum + 1;
        }
        catch (Exception e) {
            sum = sum + 10;
        }
        ❻ finally {
            ❼ sum = sum + 100;
        }
        ❽ System.out.print(sum);
    }

    ❹ static void func( ) throws Exception {
        ❺ throw new NullPointerException( );
    }
}
```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- ❶ 정수형 변수 sum을 선언하고, 0으로 초기화한다.
- ❷ 예외 구문의 시작이다.
- ❸ func() 메소드를 호출한다. ❹번으로 이동한다.
- ❹ 강제로 예외를 발생시키는 func() 메소드의 시작점이다.
- ❺ NullPointerException 예외를 강제로 발생시킨다. 예외가 발생되었으므로 try 블록이 종료되고 catch 블록으로 제어가 이동된다. ❻번으로 이동한다.

※ **throw** : 강제로 예외를 발생시키는 키워드

※ **NullPointerException** : Null을 가진 객체를 참조하는 경우 사용하는 예외 객체

⑥ **NullPointerException**에 해당하는 예외를 다루는 **catch**문의 시작이다.

⑦ **sum**에 1을 더한다. **sum**은 1이다. **try~catch**문이 종료되었으므로 ⑧번으로 이동한다.

⑧ **try~catch**문이 모두 종료되면 실행되는 **finally**문의 시작이다.

⑨ **sum**에 100을 더한다. **sum**은 101이다.

⑩ **sum**의 값 101을 출력한다.

결과 101

[문제 18]

B0

※ **답안 작성 시 주의 사항** : C, Java, Python 등의 프로그래밍 언어에서는 대소문자를 구분하기 때문에 출력 결과도 대소문자를 구분하여 정확하게 작성해야 합니다. 예를 들어, 소문자 **b0**으로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 간주됩니다.

[해설]

이 문제에는 메소드에서 사용할 자료형이 메소드를 선언하는 시점이 아니라 호출하는 시점에서 정해지는 제너릭(Generic) 기법이 적용되었습니다. 제너릭 기법은 컴파일 과정에서만 적용되므로, 실행 과정에서는 컴파일 때 결정된 자료형이 제거되고 기본 자료형인 **Object** 형으로 변환되는 타입 소거(Type Erasure)가 발생합니다. 따라서 컴파일 과정에서 제너릭 **Collection** 클래스의 자료형이 **Integer**로 결정되어 ⑦번에서 호출되는 메소드가 **print(Integer a)**라고 생각할 수 있지만 실행 과정에서는 타입 소거로 인해 제너릭 **Collection** 클래스의 자료형이 **Object**로 인식되므로 **print(Object a)**가 호출되어 **B0**이 출력됩니다.

```

class Printer {                                // Printer 클래스를 정의한다.
  ① void print(Integer a) {
      System.out.print("A" + a);
    }
  ② void print(Object a) {
      ③ System.out.print("B" + a);
    }
  ④ void print(Number a) {
      System.out.print("C" + a);
    }
}

public class Main {
    public static void main(String[] args) {
      ① new Collection<>().print( );
    } ②
    ③ public static class Collection<T> {
      ④ T value;
      ⑤ public Collection(T t) {
      ⑥     value = t;
      ⑦     }
      ⑧ public void print( ) {
      ⑨     new Printer( ).print(value);
      ⑩     }
    }
}

```

① 반환값이 없는 메소드 print(Integer a)를 정의한다. 인수가 Integer 형일 때 호출된다.

② 반환값이 없는 메소드 print(Object a)를 정의한다. 인수가 Object 형일 때 호출된다.

④ 반환값이 없는 메소드 print(Number a)를 정의한다. 인수가 Number 형일 때 호출된다.

※ ①, ②, ④ 세 개의 메소드는 이름은 같지만 인수의 자료형이 다르므로 서로 다른 메소드입니다. 이렇게 이름은 같지만 인수를 받는 자료형을 달리하여 여러 기능을 정의하는 것을 오버로딩(Overloading)이라고 합니다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

① 제너릭 클래스인 Collection 클래스를 사용하여 print() 메소드를 호출한다. 먼저 0을 인수로 Collection 클래스를 호출하여 생성자를 실행한 후 Collection 클래스의 print() 메소드를 호출한다. ②번으로 이동한다.

※ 제너릭(Generic) : 메소드에서 사용할 자료형을 메소드를 선언할 때가 아니라 생성하거나 호출할 때 정하는 기법으로, new Collection<>().print();와 같이 클래스 선언문에 홑화살괄호(< >)가 사용된 것으로 제너릭임을 알 수 있습니다.

② 제너릭 클래스의 시작점이다. ①번에서 전달한 0이 정수이므로, 제너릭 클래스의 제너릭 자료형 T는 Integer로 결정된다. 하지만 이는 컴파일 과정에서만 유효한 것이고 실행 과정에서는 기본자료형인 Object로 취급된다.

③ 'Object value'와 동일하다. Object형 변수 value를 선언한다.

④ 제너릭 Collection 클래스의 생성자이다. 자료형 T는 Object이며, ①번에서 전달한 0을 t가 받는다.

public Collection(T t)

· T : 자료형으로, 호출될 때의 자료형으로 자동 지정됨

· t : 인수

- ⑤ value에 t의 값 0을 저장한다.
- ⑥ 제너릭 Collection 클래스의 print() 메소드의 시작점이다.
- ⑦ Printer 객체를 생성하고, value를 인수로 print() 메소드를 호출한다. value의 자료형이 Object이므로 Printer 클래스의 print(Object a) 메소드를 호출한다. ⑧번으로 이동한다.
- ⑧ 반환값이 없는 print(Object a) 메소드의 시작점이다. ⑦번에서 전달한 value를 a가 받는다.
- ⑨ a의 값 0 앞에 문자 "B"를 덧붙여 출력한다. 메소드가 종료되면, 호출했던 ⑩번으로 이동한다.

결과 B0

- ⑩ 메소드가 종료되면, 호출했던 ⑪번으로 이동한 후 프로그램을 종료한다.

[문제 19]

1

[해설]

```
#include <stdio.h>
⑥ void func(int** arr, int size) {
⑦     for(int i = 0; i < size; i++) {
⑧         *(*arr + i) = (*(*arr + i) + i) % size;
            }
    } ⑨

int main( ){
①     int arr[] = {3, 1, 4, 1, 5};
②     int* p = arr;
③     int** pp = &p;
④     int num = 6;
⑤     func(pp, 5);
⑩    num = arr[2];
⑪    printf("%d", num);
⑫    return 0;
}
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

- ① 5개의 요소를 갖는 정수형 배열 arr을 선언한다. 이후 그림에서 지정한 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.

메모리

주소					
0000					
:					
	첫 번째	두 번째	세 번째	네 번째	다섯 번째
arr 0500	3	1	4	1	5
	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
:					
1000					
:					
2000					
:					
9999					

- ② 정수형 포인터 변수 p를 선언하고, arr의 주소로 초기화한다. p는 arr의 시작 주소를 가리킨다.

메모리

주소	0000				
:	:				
		첫 번째	두 번째	세 번째	네 번째
arr	0500	3	1	4	1
		arr[0]	arr[1]	arr[2]	arr[3]
:	:				
p	1000	500			
:	:				
	2000				
:	:				
	9999				

- ③ 정수형 이중포인터 변수 pp를 선언하고, 포인터 변수 p의 주소를 저장한다. p가 arr의 시작 주소를 가리키고 있으므로, p의 주소를 가리키는 pp도 arr의 시작 주소를 가리킨다.

메모리

주소	0000				
:	:				
		첫 번째	두 번째	세 번째	네 번째
arr	0500	3	1	4	1
		arr[0]	arr[1]	arr[2]	arr[3]
:	:				
p	1000	500			
:	:				
pp	2000	1000			
:	:				
	9999				

- ④ 정수형 변수 num을 선언하고, 초기값으로 6을 저장한다.
- ⑤ pp와 5를 인수로 func() 함수를 호출한다. ⑥번으로 이동한다.
- ⑥ 반환값이 없는 func() 함수의 시작점이다. ⑤번에서 전달한 pp를 arr이, 5를 size가 받는다.
- ⑦ 반복 변수 i가 0부터 1씩 증가하면서 size보다 작은 동안 ⑧번을 반복 수행한다.
- ⑧ $*(arr + i) = (*(arr + i) + i) \% size$ 는 $arr[i] = (arr[i] + i) \% size$ 와 동일하다.
- ***arr** : 이중포인터 변수 arr에는 main() 함수에서 전달된 이중포인터 변수 pp가 저장되어 있으므로, arr이 가리키는 곳은 arr 배열의 시작 주소이다.
 - ***arr + i** : arr 배열의 시작 주소에 i를 더한다는 것은 arr 배열의 i 번째 요소를 의미하므로, arr[i]와 같다.
 - ***(arr + i)** : arr[i]의 값을 의미한다.
- ※ 반복문의 실행에 따른 변수들의 변화는 다음과 같다.

size	i	arr[i]	arr[i]+i	(arr[i]+i) % size	arr 배열
5	0	3	3	3	3 1 4 1 5 3
	1	1	2	2	3 1 4 1 5 2
	2	4	6	1	3 2 4 1 5 1
	3	1	4	4	3 2 1 1 5 4
	4	5	9	4	3 2 1 4 5 4
	5				3 2 1 4 4

⑨ func() 함수가 종료되면, func() 함수를 호출했던 ⑩번으로 제어를 옮긴다.

⑩ arr[2]의 값 1을 num에 저장한다.

⑪ num의 값 1을 출력한다.

결과 1

⑫ main() 함수에서의 'return 0'은 프로그램의 종료를 의미한다.

[문제 20]

㉠